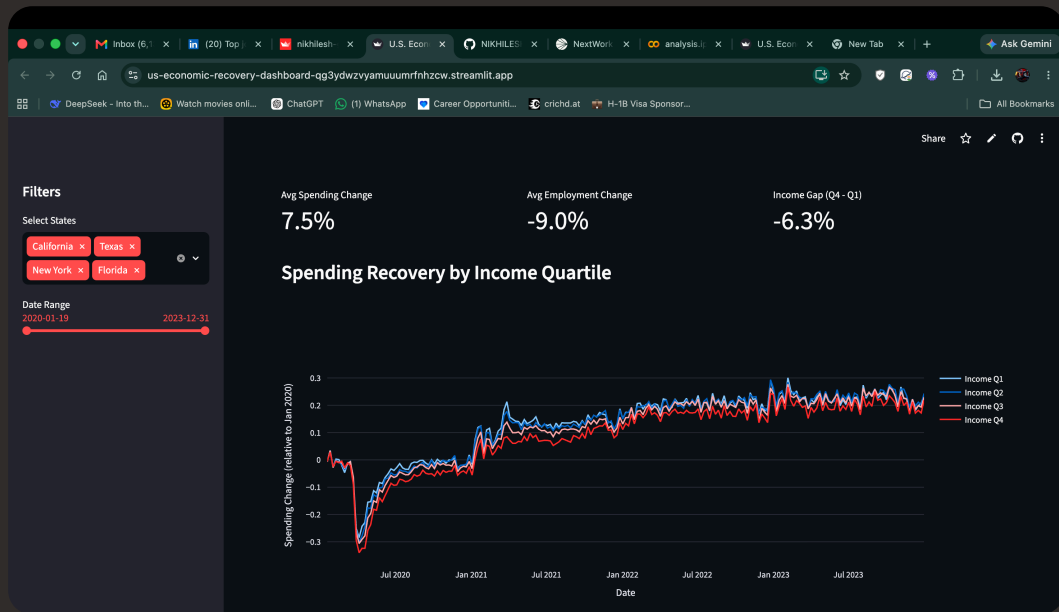




U.S. ECONOMIC RECOVERY DASHBOARD

By Nikhil Goud | June 2026



Building a Live Economic Recovery Dashboard

Project vision and goals

In this project, we are analyzing how the U.S. economy recovered after the COVID-19 shutdowns in 2020. The main focus is to understand how consumer spending and employment changed across different U.S. states and income groups.

We will use real economic datasets from the Harvard Opportunity Insights Economic Tracker. These datasets include information about spending patterns, employment changes, income quartiles, and state-level economic activity.

The goal of this project is to clean, combine, and analyze multiple CSV files, then build an interactive dashboard using Streamlit. The dashboard will help users explore economic recovery trends by state, income group, and time period.

By the end of this project, we will have:

- A cleaned and prepared dataset
- A documented analysis notebook
- Exploratory data analysis with charts
- A live Streamlit dashboard
- A GitHub repository with complete documentation
- A state-by-state economic resilience scoring system

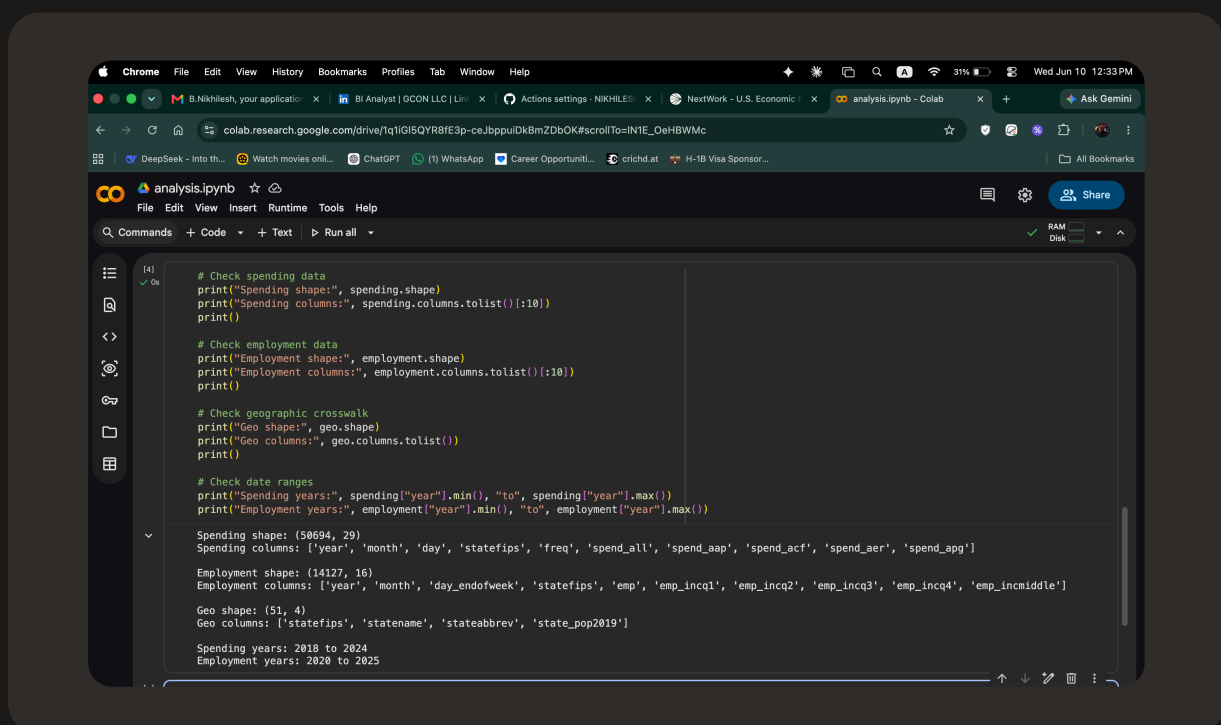
Loading Real-World Harvard Economic Data

Environment and data setup

In this step, I set up the project workspace for the ****U.S. Economic Recovery Dashboard****. I created a public GitHub repository to store the analysis notebook, Streamlit dashboard code, documentation, data files, requirements, screenshots, and deployment details.

I also opened a Google Colab notebook to explore the data in a Python environment. Then, I loaded the Harvard Opportunity Insights Economic Tracker datasets into the notebook and verified that the files were imported correctly. To confirm the data was ready, I reviewed the dataset structure, checked the available columns, viewed sample records, and confirmed the date ranges. This ensured the data was properly loaded and prepared for the next steps of cleaning, wrangling, exploratory analysis, visualization, and dashboard development.

By the end of this step, the GitHub repository and Google Colab workspace were ready, and the economic data was successfully loaded and verified.



DataFrames loaded and verified

In this step, I loaded three CSV files directly from the Harvard Opportunity Insights Economic Tracker GitHub repository into Google Colab. The datasets included daily consumer spending data by state, weekly employment data by state, and a geographic crosswalk file that maps state FIPS codes to state names.

The spending dataset tracks consumer spending changes compared to a January 2020 baseline, while the employment dataset tracks employment changes over time. The geo file helps convert numeric state codes into readable state names for analysis and visualization.

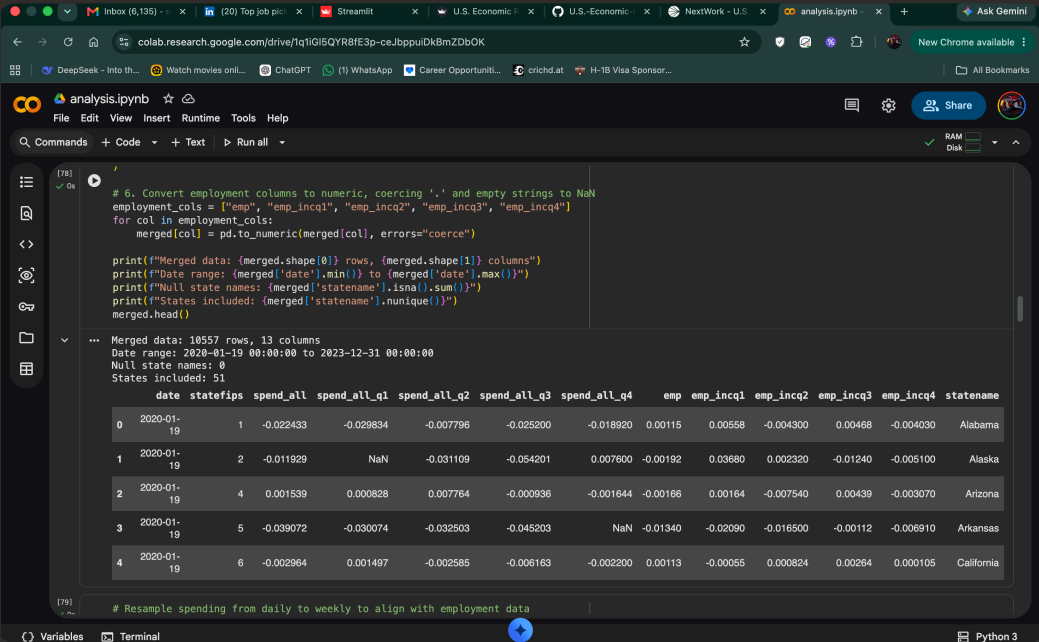
After loading the files with Pandas, I verified that all three DataFrames loaded correctly by checking their shapes, column names, and year ranges. I confirmed that the spending data included columns such as statefips, year, month, day, and spend_all, while the employment data included statefips, year, month, day_endofweek, and emp. This confirmed the data was ready for cleaning and analysis.

Wrangling and Merging Multi-File Datasets

Cleaning and aligning the data

In this step, I cleaned and prepared the raw Economic Tracker datasets for analysis. The spending and employment files had different time frequencies, with spending reported daily and employment reported weekly, so I aligned them into a consistent weekly format.

I filtered both datasets to focus on the 2020–2023 recovery period. I also handled messy values such as dots, empty strings, and missing data by converting key columns into proper numeric formats. Date fields were created from the year, month, and day columns so the data could be analyzed over time. After cleaning, I merged the spending and employment datasets using state FIPS codes and dates. I also joined the geographic crosswalk file to add readable state names. The final result was one analysis-ready DataFrame containing weekly state-level spending, employment, income quartile spending, and state name information.



```
[78]: # 6. Convert employment columns to numeric, coercing '.' and empty strings to NaN
employment_cols = ["emp", "emp_incq1", "emp_incq2", "emp_incq3", "emp_incq4"]
for col in employment_cols:
    merged[col] = pd.to_numeric(merged[col], errors="coerce")

print(f"Merged data: {merged.shape[0]} rows, {merged.shape[1]} columns")
print(f"Date range: {merged['date'].min()} to {merged['date'].max()}")
print(f"Null state names: {merged['statename'].isna().sum()}")
print(f"States included: {merged['statename'].nunique()}")
merged.head()
```

... Merged data: 10557 rows, 13 columns
Date range: 2020-01-19 00:00:00 to 2023-12-31 00:00:00
Null state names: 0
States included: 51

	date	statefips	spend_all	spend_all_q1	spend_all_q2	spend_all_q3	spend_all_q4	emp	emp_incq1	emp_incq2	emp_incq3	emp_incq4	statename
0	2020-01-19	1	-0.022433	-0.029834	-0.007796	-0.025200	-0.018920	0.00115	0.00558	-0.004300	0.00468	-0.004030	Alabama
1	2020-01-19	2	-0.011929	NaN	-0.031109	-0.054201	0.007600	-0.00192	0.03680	0.002320	-0.01240	-0.005100	Alaska
2	2020-01-19	4	0.001539	0.000828	0.007764	-0.000936	-0.001644	-0.00166	0.00164	-0.007540	0.00439	-0.003070	Arizona
3	2020-01-19	5	-0.039072	-0.030074	-0.032503	-0.045203	NaN	-0.01340	-0.02090	-0.018500	-0.00112	-0.006910	Arkansas
4	2020-01-19	6	-0.002964	0.001497	-0.002585	-0.006163	-0.002200	0.00113	-0.00055	0.000824	0.00264	0.000105	California

```
[79]: # Resample spending from daily to weekly to align with employment data
```

Merged DataFrame structure and meaning

In this step, I cleaned and merged the raw spending and employment datasets into one analysis-ready DataFrame. First, I created proper date columns for both datasets. The spending file used year, month, and day columns, while the employment file used year, month, and day_endofweek.

I filtered both datasets to include only the 2020–2023 recovery period and selected the key spending and employment columns needed for analysis. Since spending data was daily and employment data was weekly, I converted spending into weekly averages using state and date grouping.

After aligning the time frequency, I merged weekly spending with employment using statefips and date. I then joined the GeoIDs file to add readable state names. Finally, I converted numeric columns properly so missing values like dots or blanks became NaN. This produced a clean dataset ready for analysis and dashboard visuals.

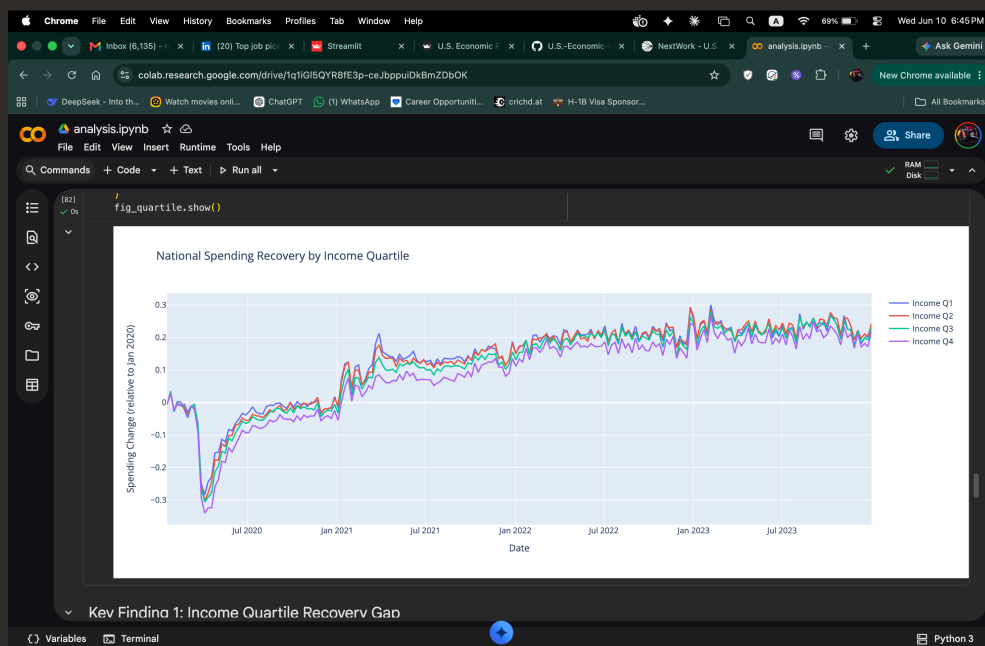
Uncovering Income Inequality in Economic Recovery

Analytical approach

In this step, I performed exploratory analysis on the cleaned economic recovery dataset to identify meaningful patterns across U.S. states and income groups. I calculated key recovery metrics such as average consumer spending recovery, employment recovery trends, and the gap between low-income and high-income spending groups.

I then created interactive Plotly visualizations to make the analysis easier to understand. The charts show how spending changed over time, how employment recovered across states, and how recovery differed between income quartiles. These visuals help highlight the uneven nature of the economic recovery after 2020.

This step connects the data analysis to a business narrative by showing which states recovered more strongly, which areas showed slower employment improvement, and how income inequality affected recovery patterns. These insights will be used as the foundation for the Streamlit dashboard.



Key findings from the visualizations

Based on the visualizations, the highest-income group, Q4, recovered the fastest. Their spending returned to the January 2020 baseline earlier than the lower-income groups and stayed stronger through much of the recovery period. In contrast, Q1, the lowest-income group, showed a slower and weaker recovery, which highlights an income-based recovery gap.

One state-level pattern I noticed is that spending recovery and employment recovery did not always move together. Some states showed stronger consumer spending even when employment recovery was still weaker, while others had better job recovery but slower spending improvement. This suggests that economic recovery should not be measured using only one metric. Looking at both spending and employment gives a clearer picture of each state's overall recovery strength.

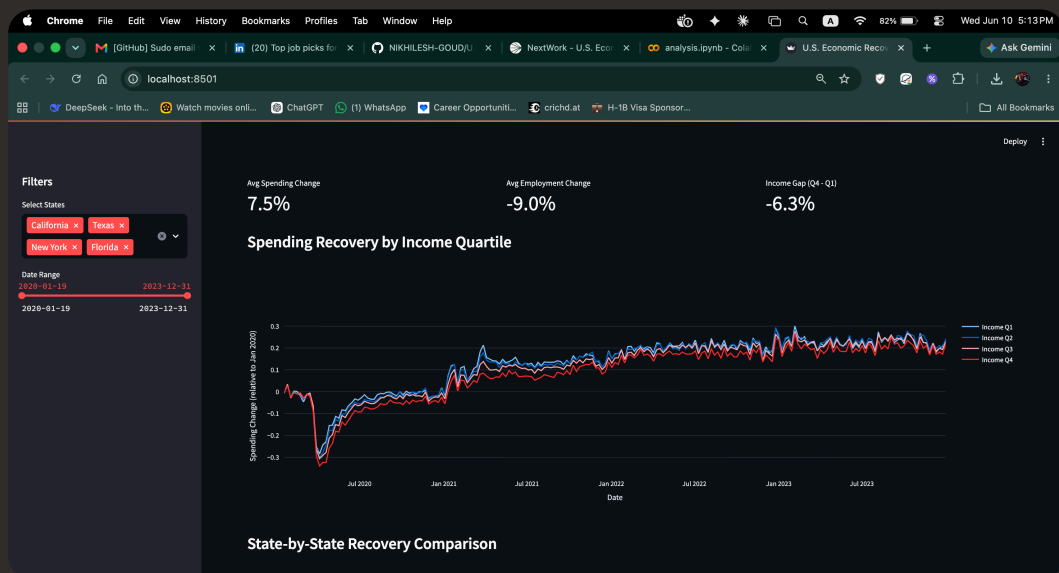
Building an Interactive Streamlit Dashboard

Dashboard architecture and components

In this step, I converted the analysis notebook into an interactive Streamlit dashboard. I created an `app.py` file to organize the dashboard logic, including data loading, page layout, sidebar filters, KPI metrics, and Plotly visualizations.

The dashboard allows users to explore the cleaned economic recovery data by selecting a state and date range. Based on the selected filters, the app updates key metrics such as average spending recovery, average employment recovery, and income quartile recovery gaps. It also displays interactive charts that show spending trends, employment trends, and the relationship between spending and employment recovery.

I reused the insights from the notebook and made them accessible through a live web interface. I also created a `requirements.txt` file with the required Python dependencies so the app can be installed, run, and deployed consistently.



Interactive filter behavior

When the state selection is changed in the Streamlit sidebar, the dashboard automatically updates based on the selected states. The app filters the main dataset to include only records for those states and the selected date range. As a result, the KPI metrics update immediately to show the average spending change, average employment change, and income gap for the selected states. This makes the dashboard interactive because users can compare different states without changing the code.

The state filter helps users explore how economic recovery varied across locations. For example, selecting California, Texas, New York, and Florida shows recovery patterns for large states, while selecting fewer states gives a more focused comparison. Streamlit reruns the script whenever the sidebar input changes, so the charts and metrics refresh automatically.

Deploying a Live Public Dashboard

Deployment process

In this step, I prepared the project for public sharing and deployment. I updated the GitHub repository with a professional README.md that explains the project objective, business question, dataset source, methodology, dashboard features, key findings, and deployment details.

The README helps hiring managers and reviewers quickly understand what the project does, why it matters, and how the analysis was performed. It also documents the tools used, including Python, Pandas, Plotly, Streamlit, Google Colab, and GitHub.

After finalizing the repository files, I deployed the Streamlit dashboard to Streamlit Community Cloud. This created a public URL where users can interact with the dashboard without running the code locally.

Finally, I verified that the live dashboard opened successfully, loaded the data correctly, displayed the filters, KPI metrics, and charts, and worked as expected from the public link.

Live URL and verified functionality

My live dashboard URL is:

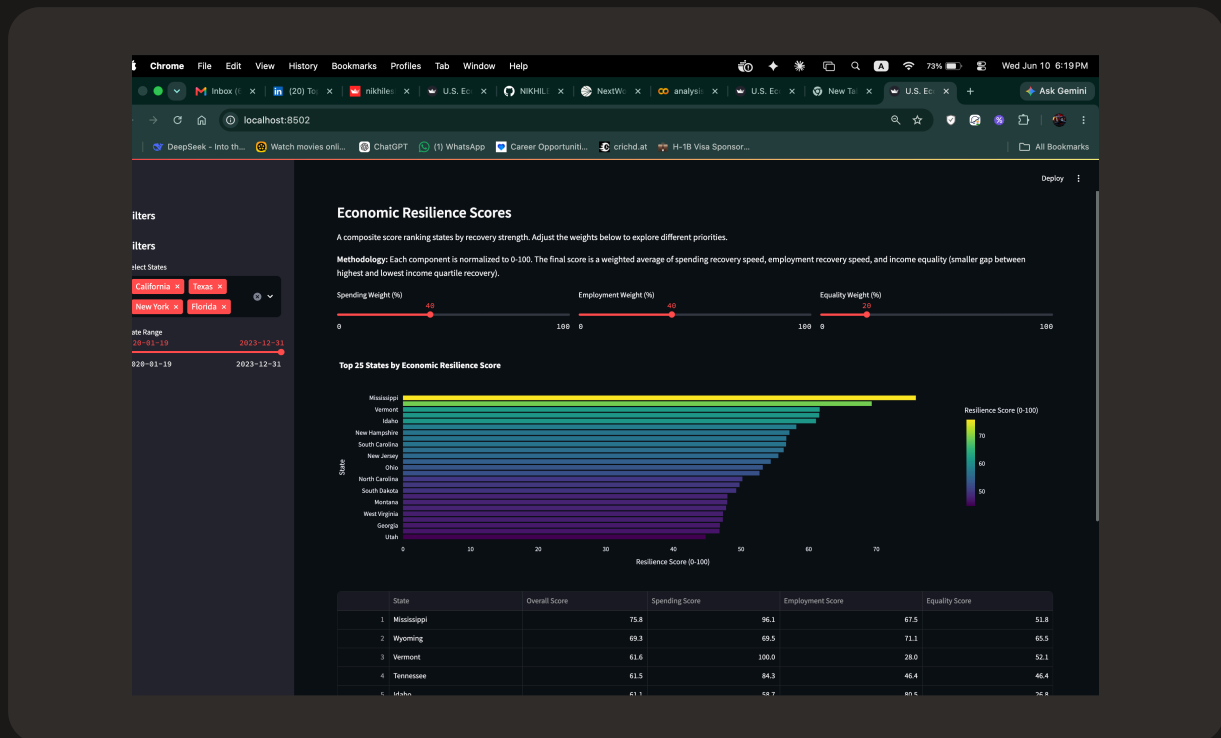
<https://us-economic-recovery-dashboard-qg3ydwzvyamuumrfnhzcw.streamlit.app>

I verified that the deployed Streamlit dashboard opens successfully from the public URL. The sidebar filters are visible, including the state multiselect and date range slider. The default states California, Texas, New York, and Florida are selected correctly.

I also verified that the KPI metrics load at the top of the dashboard, including average spending change, average employment change, and the income gap between Q4 and Q1. The dashboard shows the Recovery Analysis and Resilience Scores tabs, confirming that the app structure deployed correctly.

The public app is now accessible without running the code locally, which makes it easy for hiring managers or reviewers to explore the project directly from the live link.

Adding a Dynamic Economic Resilience Scoring System



How adjustable weights reveal different recovery narratives

Adjusting the weight sliders changes how much importance each recovery factor has in the final state ranking. The resilience score combines spending recovery, employment recovery, and income equality into one weighted score. When the spending weight is increased, states with stronger consumer spending recovery rank higher. When the employment weight is increased, states with stronger job recovery move up. When the equality weight is increased, states with smaller gaps between Q4 and Q1 spending recovery rank higher.

This matters because economic recovery can be interpreted in different ways. A state may look strong if only spending is considered, but weaker if employment recovery or income inequality is included. The sliders allow users to test different priorities and see how rankings change. This makes the dashboard more useful because it shows that recovery is not one-dimensional; a balanced interpretation should consider growth, jobs, and fairness together.

Reflections and Takeaways

Tools and concepts mastered

In this project, I learned how to build an end-to-end data analytics dashboard using free cloud-based tools. I used GitHub to store and manage the project files, Google Colab to clean and analyze the data, and Streamlit Community Cloud to deploy the dashboard as a public web app. I worked with real-world economic data from the Harvard Opportunity Insights Economic Tracker and learned how to clean messy CSV files, handle missing values, align daily and weekly datasets, and merge multiple DataFrames using state FIPS codes.

I also learned how to calculate recovery metrics, compare spending and employment trends, and analyze income-based recovery gaps. Using Plotly, I created interactive charts, and with Streamlit, I added filters, KPI cards, tabs, and dynamic resilience score sliders. The key concepts I practiced were data wrangling, exploratory data analysis, interactive visualization, dashboard development, GitHub documentation and communicating business insights from data.

Time and challenges

It took me approximately 120 minutes to complete this project. During this time, I set up the GitHub repository, loaded and cleaned the Opportunity Insights Economic Tracker datasets, performed exploratory analysis, created interactive Plotly visualizations, built the Streamlit dashboard, added the resilience scoring feature, deployed the app to Streamlit Community Cloud, and documented the project with a professional README.

I also faced some difficulty while launching the Streamlit app because of issues related to metric formatting and the Python/package version compatibility. I resolved these issues by reviewing the deployment errors, updating the required dependencies, and confirming that the dashboard worked correctly after redeployment.

I did this project today to learn how to build an end-to-end data analytics portfolio project using real-world economic data. I practiced loading and cleaning CSV files, merging spending and employment datasets, creating recovery metrics, building interactive Plotly charts, and deploying a Streamlit dashboard to a public URL.

This project also helped me understand how to present technical work in a way that is useful for recruiters and hiring managers. Instead of only writing code, I documented the business question, methodology, findings, dashboard features, and live deployment link so reviewers can quickly understand the value of the project.

Another skill I want to learn is how to make dashboards more professional by improving UI design, adding maps, advanced filters, better storytelling, and clearer business recommendations that make the analysis stand out in a job portfolio.